

Labwork 10.2 - Recursive & Bitwise

General Information and Submission Rules:

- By the end of the labwork, you must submit it to Yulearn as **studentID_fullname_labwork10.c**.
- Only use concepts covered in the labs so far.

Question 1 - Display Message (10p)

- Create a new function (**greetings**) that displays a welcome message and a list of all available operations to the user.
- This function will expect a decision from a user so that it will execute related functions based on this choice until the user exits.

```
Welcome to the Number Games Machine! By using this machine you can do:
1. The Anagram Decryption
2. Reverse Array
3. Even / Odd
4. Addition
5. Exit
Make your choice:
```

Question 2 - The Anagram Decryption (25p)

- A door is locked with a 4-letter password. The system accepts inputs by trying every single letter arrangement.
- Write a recursive function (**anagram_decryption**) that takes a string and prints all possible permutations of its characters.
- You can write a helper function (**string_length**) to find out the size of the string and add it as a parameter to the function.
- Define a random initial string in main and use it as an argument to the function.

```
Main:
```

```
char key[] = "CODE";  
generatePasswords(key); //You can add additional parameters
```

Output:

```
>> 1. CODE  
>> 2. COED  
>> 3. CDOE  
>> 4. ...
```

Question 3 - Reverse Array (15p)

- An anomaly is causing incoming text packets to stream in reverse order. There are memory limits to allocate a secondary array, so reconstruction must happen entirely within the original memory space.
- Write a recursive function (**reverse_packet**) that reverses a character array (string) in place.
- You can write a helper function (**string_length**) to find out the size of the string and add it as a parameter to the function.
- Define a random initial string in main and use it as an argument to the function.

Main:

```
char packet[] = "ANOMALY";  
reverse_packet(packet); //You can add additional parameters
```

Output:

```
>> YLAMONA
```

Question 4 - Even / Odd (25p)

- A factory sorts packages into a "left" bin (even IDs) and a "right" bin (odd IDs). You cannot use % operator.
- Write a function (**bitwise_even_odd**) that determines if an integer is even or odd using a single bitwise operation.

Main:

```
int package = 11;  
bitwise_even_odd(package);
```

Output:

```
>> 11 -> Odd
```

Question 5 - Addition (25p)

- Write a function (**bitwise_add**) to add two integers using **only** bitwise operators and loops.

Main:

```
int a = 5;  
int b = 10;  
int result = bitwise_add(a, b);
```

Output:

```
5 + 10 = 15
```